

"DEVELOPMENT OF A LEXICAL ANALYZER"

Problem Statement

1. Develop a lexical analyzer (LA) program for a subset of the SIGNAL programming language.
2. The lexical analyzer must perform the following actions:
 - removing (skipping) whitespace characters: space (ASCII code 32), carriage return (ASCII code 13); line feed (ASCII code 10), horizontal and vertical tabulation (ASCII codes 9 and 11), form feed (ASCII code 12);
 - tokenizing keywords;
 - tokenizing multi-character delimiters (if provided by the variant's grammar);
 - tokenizing constants, recording the constant's value and type into a table (if provided by the variant's grammar);
 - tokenizing identifiers;
 - removing comments specified in the format (**comment text**);
 - generating a stream of tokens with information about token positions;
 - populating the identifier and constant tables with information obtained during tokenization;
 - outputting error messages.

The work is performed according to **Variant 12**.

Grammar according to the variant:

Variant 12

1. `<signal-program> --> <program>`
2. `<program> --> PROCEDURE <procedure-identifier>
 <parameters-list> ; <block> ;`
3. `<block> --> <declarations> BEGIN <statements-list>
 END`
4. `<declarations> --> <label-declarations>`
5. `<label-declarations> --> LABEL <unsigned-integer>
 <labels-list>; |
 <empty>`
6. `<labels-list> --> , <unsigned-integer> <labels-list>
 |
 <empty>`
7. `<parameters-list> --> (<variable-identifier>
 <identifiers-list>) |
 <empty>`
8. `<identifiers-list> --> , <variable-identifier>
 <identifiers-list> |
 <empty>`
9. `<statements-list> --> <statement> <statements-list>
 |
 <empty>`
10. `<statement> --> <unsigned-integer> : <statement> |
 GOTO <unsigned-integer> ; |
 RETURN ; |
 ; |
 ($ <assembly-insert-file-identifier> $)`
11. `<variable-identifier> --> <identifier>`
12. `<procedure-identifier> --> <identifier>`
13. `<assembly-insert-file-identifier> --> <identifier>`
14. `<identifier> --> <letter><string>`
15. `<string> --> <letter><string> |
 <digit><string> |
 <empty>`
16. `<unsigned-integer> --> <digit><digits-string>`
17. `<digits-string> --> <digit><digits-string> |
 <empty>`
18. `<digit> --> 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9`
19. `<letter> --> A | B | C | D | ... | Z`

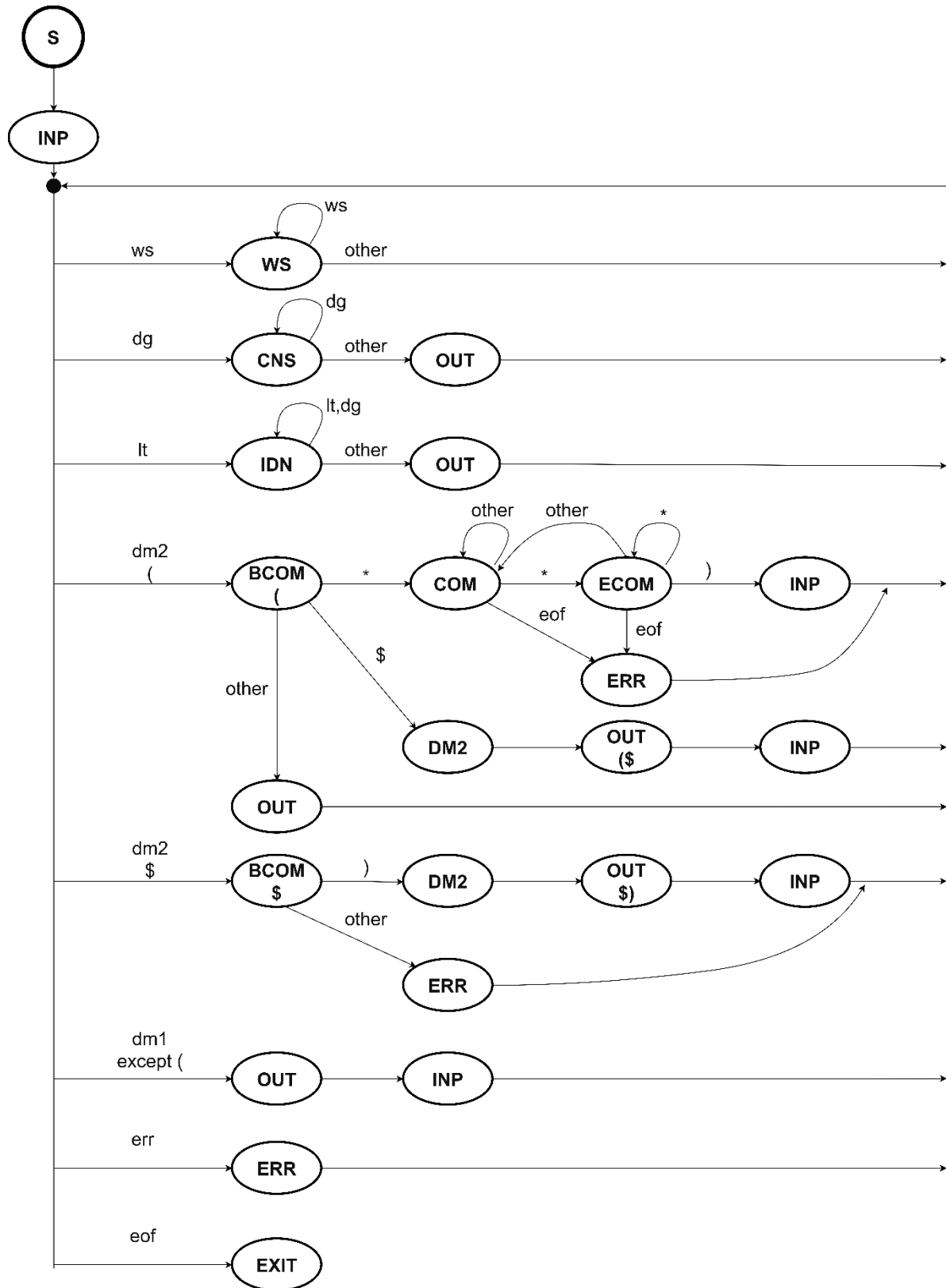
Automaton graph:

The meaning of the states in the partial subgraphs for extracting and processing the aforementioned tokens:

- INP – reading the next character;
- OUT – processing and outputting the lexeme (token);
- NUM – processing the current character of the '**number**' token and reading the next character;
- IDN – processing the current character of the '**identifier**' token and reading the next character;
- DM2 – reading the next character after the '<' character, which is the starting character of a potential two-character delimiter '<=', and making a decision about the next transition;
- WS – skipping the current whitespace character and reading the next character;
- BCOM – reading the next character after the first character of the comment start and making a decision about the next transition;
- COM – reading the next character of the comment and making a decision about the next transition;
- ECOM – reading the next character after the '*' character and making a decision on whether to terminate or continue the comment;
- ERR – issuing an error message.

In all states, except OUT and ERR, the next character is read. In the OUT state, in addition to outputting the generated token (code, line, column) to a file (token stream), all necessary actions with the corresponding tables of identifiers, constants, etc., are also performed, while the next character—which has already been read but not yet analyzed—is passed forward for further processing.

The label '**other**' is also used on the edges of the LA automaton's partial subgraphs, which means that the transition occurs upon receiving any character other than those whose categories are present on the other edges originating from the same state.



Test cases:

1. True-test:

Input program:

```
PROCEDURE TRUETEST(A, B, C);
BEGIN
  (* Comment:
  (****) $)
D: ;: (*, $
*)
  LABEL 1, 100, 999;

  (100); (* These are the parameters in parentheses. *)

  ($ ASM START $)

10: 20: 30: ; (* Labels and separators *)

GOTO 100;
RETURN;
END;
```

Output program:

Result Lexem List:

Row	Col	Code	Lexem
1	1	401	PROCEDURE
1	11	1001	TRUETEST
1	19	40	(
1	20	1002	A
1	21	44	,
1	23	1003	B
1	24	44	,
1	26	1004	C
1	27	41	)
1	28	59	;
2	1	402	BEGIN
4	12	302	\$)
5	1	1005	D
5	2	58	:
5	4	59	;
5	6	58	:
7	5	404	LABEL
7	11	501	1
7	12	44	,
7	14	502	100
7	17	44	,
7	19	503	999
7	22	59	;
9	5	40	(
9	6	502	100
9	9	41	)
9	10	59	;
11	5	301	(\$
11	8	1006	ASM
11	12	1007	START
11	18	302	\$)
13	2	504	10
13	4	58	:
13	6	505	20
13	8	58	:
13	10	506	30
13	12	58	:
13	14	59	;
15	1	405	GOTO
15	6	502	100
15	9	59	;
16	1	406	RETURN
16	7	59	;
17	1	403	END
17	4	59	;

			Identifiers:
Code	Lexem	Type	
1001	TRUETEST	-	
1002	A	-	
1003	B	-	
1004	C	-	
1005	D	-	
1006	ASM	-	
1007	START	-	
			Constants:
Code	Lexem	Type	
501	1	unsigned-integer	
502	100	unsigned-integer	
503	999	unsigned-integer	
504	10	unsigned-integer	
505	20	unsigned-integer	
506	30	unsigned-integer	

2. False-тести:

- Test for checking invalid operators:

Input program: PROCEDURE FALSE; BEGIN (* Invalid operators and characters *) A = B + C - D * E / F; (* Special characters and punctuation *) VAL ? & # @ ! ~ ^; (* Incorrect paths or system signs *) PATH \ FILE [] { } < > . (*dollar sign (\$ \$)*) \$ END;	Output program: Lexer: Error (4,7) Illegal symbol '=' Lexer: Error (4,11) Illegal symbol '+' Lexer: Error (4,15) Illegal symbol '-' Lexer: Error (4,19) Illegal symbol '*' Lexer: Error (4,23) Illegal symbol '/' Lexer: Error (7,13) Illegal symbol '?' Lexer: Error (7,15) Illegal symbol '& Lexer: Error (7,17) Illegal symbol '#' Lexer: Error (7,19) Illegal symbol '@' Lexer: Error (7,21) Illegal symbol '!' Lexer: Error (7,23) Illegal symbol '~' Lexer: Error (7,25) Illegal symbol '^' Lexer: Error (10,6) Illegal symbol '\' Lexer: Error (10,13) Illegal symbol '[' Lexer: Error (10,15) Illegal symbol ']' Lexer: Error (10,17) Illegal symbol '{' Lexer: Error (10,19) Illegal symbol '}' Lexer: Error (10,21) Illegal symbol '<' Lexer: Error (10,23) Illegal symbol '>' Lexer: Error (10,25) Illegal symbol '.' Lexer: Error (13,1) Illegal symbol '\$' Result Lexem List: Row Col Code Lexem 1 1 401 PROCEDURE 1 11 1001 FALSE 1 16 59 ; 2 1 402 BEGIN 4 5 1002 A 4 9 1003 B
---	--

	4	13	1004	C
	4	17	1005	D
	4	21	1006	E
	4	25	1007	F
	4	26	59	;
	7	9	1008	VAL
	7	26	59	;
	10	1	1009	PATH
	10	8	1010	FILE
	14	1	403	END
	14	4	59	;
	Identifiers:			
	Code	Lexem	Type	
	1001	FALSE	-	
	1002	A	-	
	1003	B	-	
	1004	C	-	
	1005	D	-	
	1006	E	-	
	1007	F	-	
	1008	VAL	-	
	1009	PATH	-	
	1010	FILE	-	
	Constants:			
	Code	Lexem	Type	

• **Test for checking all possible comment variations:**

Input program: PROCEDURE COMMENT_TRAP; BEGIN (* Closing a comment without opening it: *) *) (* '*' ')' 3 *) LABEL 500; (* 2. Comment not closed: GOTO 500; RETURN; (* *) END;	Output program: Lexer: Error (1,18) Illegal symbol '-' Lexer: Error (4,1) Illegal symbol '*' Lexer: Error (10,5) Comment not closed '(' Result Lexem List: Row Col Code Lexem 1 1 401 PROCEDURE 1 11 1001 COMMENT 1 19 1002 TRAP 1 23 59 ; 2 1 402 BEGIN 4 2 41) 8 5 404 LABEL 8 11 501 500 8 14 59 ; Identifiers: Code Lexem Type 1001 COMMENT - 1002 TRAP - Constants: Code Lexem Type 501 500 unsigned-integer
--	--

- **Test for additional verification:**

Input program: PROCEDURE COMMENTS; BEGIN (* This is a normal comment. *) (*This is a multi-line comment *) (*****) (* Many stars *) (*(* Fake nesting *) (* Bracket inside (and one more) *) (* ; : , \$ *);\$\$ (* Dividers inside *) LABEL 10; (* A comment that ends strangely ***) GOTO 10; END;	Output program:: Lexer: Error (9,19) Illegal symbol '\$' Lexer: Error (9,20) Illegal symbol '\$' Result Lexem List: <table><tr><th>Row</th><th>Col</th><th>Code</th><th>Lexem</th></tr><tr><td>1</td><td>1</td><td>401</td><td>PROCEDURE</td></tr><tr><td>1</td><td>11</td><td>1001</td><td>COMMENTS</td></tr><tr><td>1</td><td>19</td><td>59</td><td>;</td></tr><tr><td>2</td><td>1</td><td>402</td><td>BEGIN</td></tr><tr><td>9</td><td>18</td><td>59</td><td>;</td></tr><tr><td>11</td><td>5</td><td>404</td><td>LABEL</td></tr><tr><td>11</td><td>11</td><td>501</td><td>10</td></tr><tr><td>11</td><td>13</td><td>59</td><td>;</td></tr><tr><td>14</td><td>5</td><td>405</td><td>GOTO</td></tr><tr><td>14</td><td>10</td><td>501</td><td>10</td></tr><tr><td>14</td><td>12</td><td>59</td><td>;</td></tr><tr><td>15</td><td>1</td><td>403</td><td>END</td></tr><tr><td>15</td><td>4</td><td>59</td><td>;</td></tr></table> Identifiers: <table><tr><th>Code</th><th>Lexem</th><th>Type</th></tr><tr><td>1001</td><td>COMMENTS</td><td>-</td></tr></table> Constants: <table><tr><th>Code</th><th>Lexem</th><th>Type</th></tr><tr><td>501</td><td>10</td><td>unsigned-integer</td></tr></table>	Row	Col	Code	Lexem	1	1	401	PROCEDURE	1	11	1001	COMMENTS	1	19	59	;	2	1	402	BEGIN	9	18	59	;	11	5	404	LABEL	11	11	501	10	11	13	59	;	14	5	405	GOTO	14	10	501	10	14	12	59	;	15	1	403	END	15	4	59	;	Code	Lexem	Type	1001	COMMENTS	-	Code	Lexem	Type	501	10	unsigned-integer
Row	Col	Code	Lexem																																																																		
1	1	401	PROCEDURE																																																																		
1	11	1001	COMMENTS																																																																		
1	19	59	;																																																																		
2	1	402	BEGIN																																																																		
9	18	59	;																																																																		
11	5	404	LABEL																																																																		
11	11	501	10																																																																		
11	13	59	;																																																																		
14	5	405	GOTO																																																																		
14	10	501	10																																																																		
14	12	59	;																																																																		
15	1	403	END																																																																		
15	4	59	;																																																																		
Code	Lexem	Type																																																																			
1001	COMMENTS	-																																																																			
Code	Lexem	Type																																																																			
501	10	unsigned-integer																																																																			

- **Test for additional verification:**

Input program: PROCEDURE STICKY(A,B,C); BEGIN LABEL 1,2,300; 1:2:300;; (\$ASMS\$); GOTO 1; RETURN; END.	Output program: Lexer: Error (8,4) Illegal symbol '.' Result Lexem List: <table><tr><th>Row</th><th>Col</th><th>Code</th><th>Lexem</th></tr><tr><td>1</td><td>1</td><td>401</td><td>PROCEDURE</td></tr><tr><td>1</td><td>11</td><td>1001</td><td>STICKY</td></tr><tr><td>1</td><td>17</td><td>40</td><td>(</td></tr><tr><td>1</td><td>18</td><td>1002</td><td>A</td></tr><tr><td>1</td><td>19</td><td>44</td><td>,</td></tr><tr><td>1</td><td>20</td><td>1003</td><td>B</td></tr><tr><td>1</td><td>21</td><td>44</td><td>,</td></tr><tr><td>1</td><td>22</td><td>1004</td><td>C</td></tr><tr><td>1</td><td>23</td><td>41</td><td>)</td></tr><tr><td>1</td><td>24</td><td>59</td><td>;</td></tr><tr><td>2</td><td>1</td><td>402</td><td>BEGIN</td></tr><tr><td>3</td><td>1</td><td>404</td><td>LABEL</td></tr><tr><td>3</td><td>7</td><td>501</td><td>1</td></tr><tr><td>3</td><td>8</td><td>44</td><td>,</td></tr><tr><td>3</td><td>9</td><td>502</td><td>2</td></tr><tr><td>3</td><td>10</td><td>44</td><td>,</td></tr><tr><td>3</td><td>11</td><td>503</td><td>300</td></tr><tr><td>3</td><td>14</td><td>59</td><td>;</td></tr></table>	Row	Col	Code	Lexem	1	1	401	PROCEDURE	1	11	1001	STICKY	1	17	40	(	1	18	1002	A	1	19	44	,	1	20	1003	B	1	21	44	,	1	22	1004	C	1	23	41	)	1	24	59	;	2	1	402	BEGIN	3	1	404	LABEL	3	7	501	1	3	8	44	,	3	9	502	2	3	10	44	,	3	11	503	300	3	14	59	;
Row	Col	Code	Lexem																																																																										
1	1	401	PROCEDURE																																																																										
1	11	1001	STICKY																																																																										
1	17	40	(																																																																										
1	18	1002	A																																																																										
1	19	44	,																																																																										
1	20	1003	B																																																																										
1	21	44	,																																																																										
1	22	1004	C																																																																										
1	23	41	)																																																																										
1	24	59	;																																																																										
2	1	402	BEGIN																																																																										
3	1	404	LABEL																																																																										
3	7	501	1																																																																										
3	8	44	,																																																																										
3	9	502	2																																																																										
3	10	44	,																																																																										
3	11	503	300																																																																										
3	14	59	;																																																																										

4	1	501	1
4	2	58	:
4	3	502	2
4	4	58	:
4	5	503	300
4	8	58	:
4	9	59	;
5	1	301	(\$
5	3	1005	ASM
5	6	302	\$)
5	8	59	;
6	1	405	GOTO
6	6	501	1
6	7	59	;
7	1	406	RETURN
7	7	59	;
8	1	403	END

Identifiers:

Code	Lexem	Type
------	-------	------

1001	STICKY	-
------	--------	---

1002	A	-
------	---	---

1003	B	-
------	---	---

1004	C	-
------	---	---

1005	ASM	-
------	-----	---

Constants:

Code	Lexem	Type
------	-------	------

501	1	unsigned-integer
-----	---	------------------

502	2	unsigned-integer
-----	---	------------------

503	300	unsigned-integer
-----	-----	------------------